Algorithmic Analysis

Data Structures

Chapter 3 in textbook (3.1 – 3.3)

Algorithms

- What is an ***algorithm***?

  A step-by-step procedure (of precise instructions) for performing a specified task in a finite amount of time.

  - Typically involves reading input and generating output of some sort.
- In computer science, algorithms are frequently represented using ***pseudo-code***.

  A pseudo-code is a structured description of an algorithm, typically expressed in in a high-level programming-language-like formalism, where the focus primarily is on the algorithm's execution logic.

  - Exact syntax and implementation details are (typically) ignored, thus pseudo-code is not directly executable on a computer.

Algorithms (cont.)

- What is the most important property of an algorithm?

> ***Correctness*** is the most important property of an algorithm, i.e. it does what it is supposed to do (on all legimate inputs).

- Assuming we have two (correct) algorithms for performing a given task, how do we decide if one is **"better"** than the other?

> We want an algorithm to be as ***efficient*** as possible, i.e. it runs fast (using restricted number of resourses, like CPU/memory).

  - **Efficiency** increasingly important as the size of the algorithm's input grows.

- Other properties, such as *simplicity* and *generality* (and more) also matter, but are typically of a secondary importance.

Algorithms and Data Structures

- What roles do ***data structures*** play in algorithmic design?

> Algorithms work on data, and it is generally necessary to manage the data in a specific manner for the algorithm to be able to accomplish the given task efficiently.

- Having a solid knowledge of ***data-structures*** and ***algorithms*** is ***essential*** for becoming an ***effective software developer***.

Image created using *artlist.io*

Time Complexity

- How to measure **algorithm's efficiency** (in terms of time)?

1. Measure ***running time*** when executed.
   Pros and cons:
   - Must be operational (implementation is up and running)
   - Hardware dependent

2. Count ***primitive (representative) operations*** *(in the pseudo-code)*
   Pros and cons:
   - Must define primitive operations (having constant execution time)
   - Can determine algorithms efficiency before implementing/running it
     - The number of primitive operations performed by the algorithm should be proportional to the actual running time of (an implemented) algorithm.

Time Complexity (cont.)

- How does different type of (same size) input affect performance?

- What behaviour should we measure?
  - **Best-case** behaviour
  - ***Worst-case*** behaviour
    - **most typical!**
  - **Average-case** behaviour

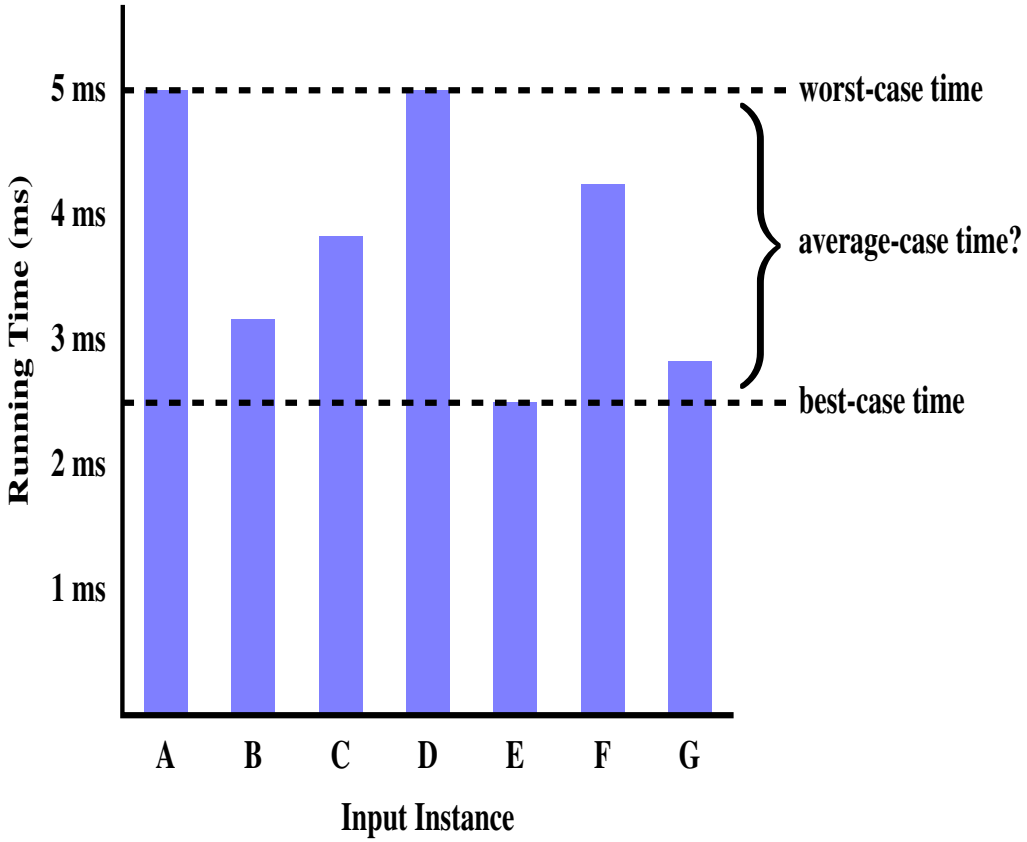

Image from the textbook: *Data Structures & Algorithms*, Goodrich et al.

Time Complexity (cont.)

- Most informative to measure number of operations as a function of input size (n)
  - $f(n)$
- Functions:
  - Constant ($f(n) = c$)
  - Logarithmic ($f(n) = log_2(n)$)
  - Linear ($f(n) = n$)
  - N-Log-N ($f(n) = n * log_2(n)$)
  - Quadratic ($f(n) = n^2$)
  - Cubic ($f(n) = n^3$)
  - Exponential ($f(n) = 2^n$)

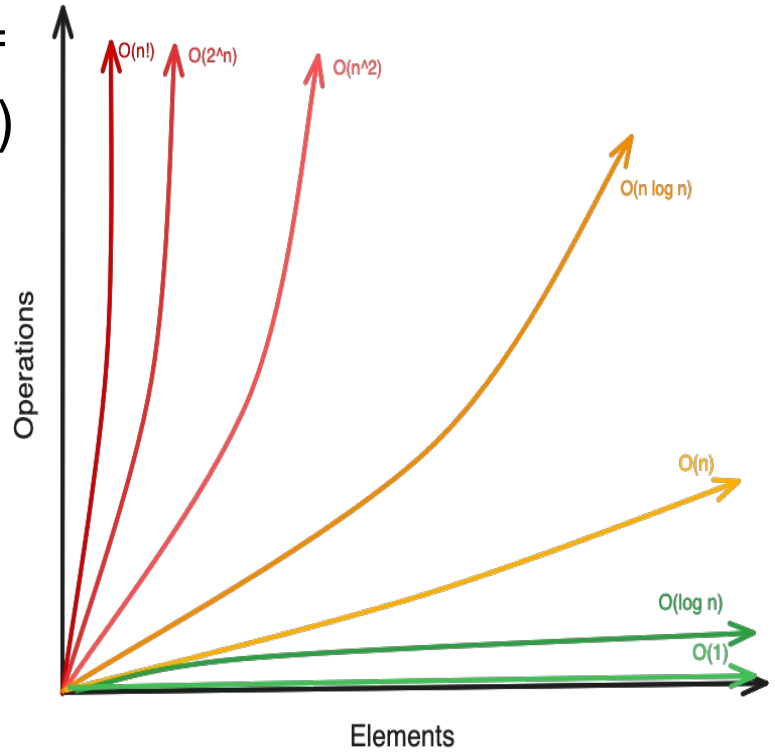

Image: https://launchschool.com

Time Complexity (cont.)

Drawn on a log scale (both *x* and *y*).

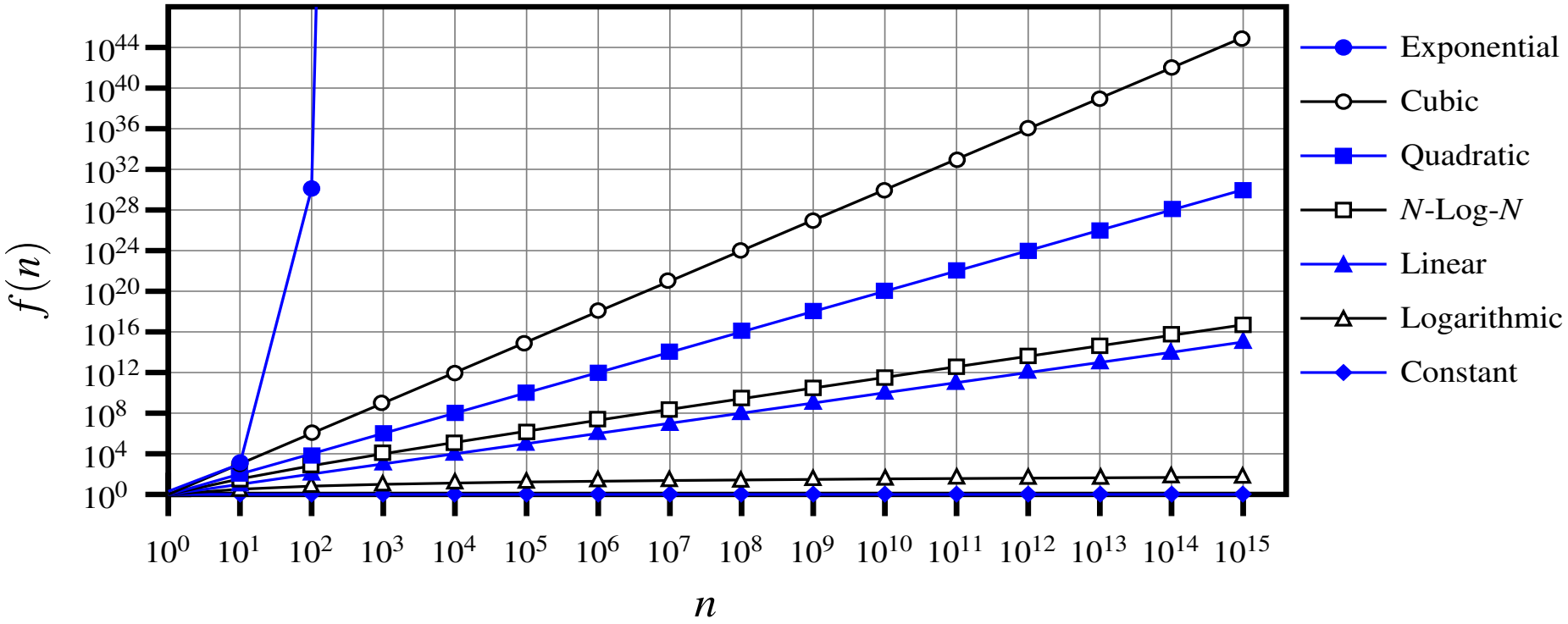

Image from the textbook: *Data Structures & Algorithms*, Goodrich et al.

Algorithms (cont.)

Algorithm 1 Bubble Sort Algorithm

Require: An array A of length n (indexed 0 to $n-1$)
Ensure: The sorted array A

```
function BubbleSort(A)
    n ← length(A)
    repeat
        swapped ← false
        for i ← 0 to n − 2 do
            if A[i] > A[i + 1] then
                swap(A[i], A[i + 1])
                swapped ← true
            end if
        end for
    until not swapped
end function
```

Algorithm 2 Selection Sort Algorithm

Require: An array A of length n (indexed 0 to $n-1$)
Ensure: The sorted array A

```
function SelectionSort(A)
    n ← length(A)
    for i ← 0 to n − 2 do
        min ← i
        for j ← i + 1 to n − 1 do
            if A[j] < A[min] then
                min ← j
            end if
        end for
        if min ≠ i then
            swap(A[i], A[min])
        end if
    end for
end function
```

Asymptotic Analysis: **Big-Oh** Notation

- We focus on the ***growth rate*** of the running time as a function of the input size *n*.
- **Big-Oh**:
  - *Let f(n)* and *g(n)* be functions mapping positive integers to real numbers.
  - We say that *f(n)* is ***O(****g(n)****)*** if there exists a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that

    $f(n) \leq cg(n)$, for $n \geq n_0$

  - Pronounced as: *f(n)* is ***big-Oh of*** *g(n)*

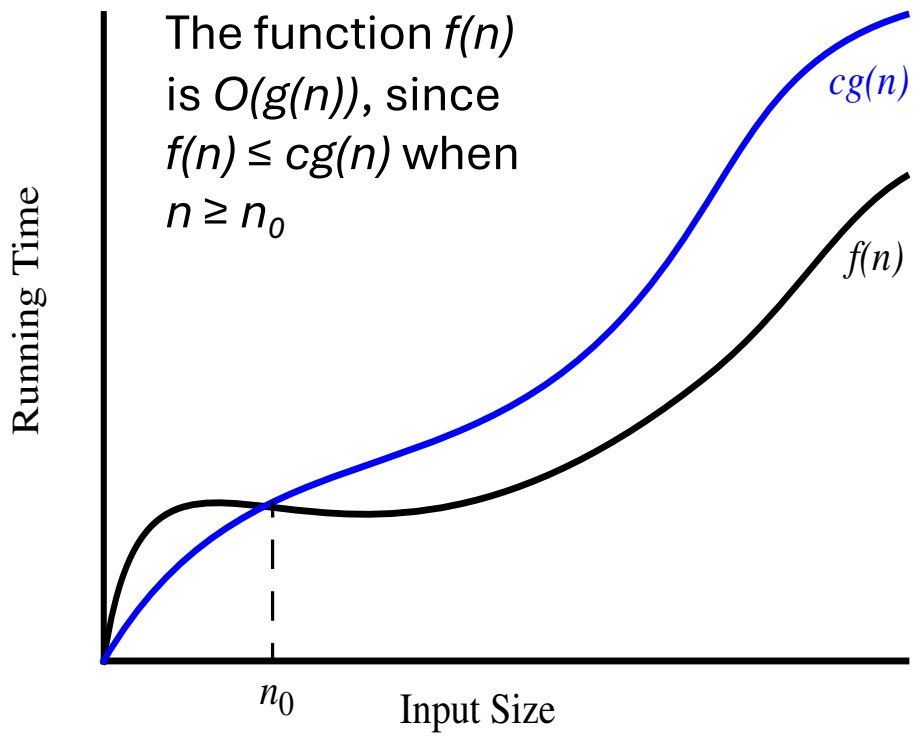

Image from the textbook: *Data Structures & Algorithms*, Goodrich et al.

Asymptotic Analysis (cont.)

- Examples:
  - $5n^4 + 3n^3 + 2n^2 + 4n + 1$ is $\mathbf{O(n^4)}$
    - *Justification:*
      - $5n^4 + 3n^3 + 2n^2 + 4n + 1 \leq (5 + 3 + 2 + 4 + 1)\, n^4 = c \times n^4$, *for* $c = 15$ *when* $n \geq n_0 = 1$
  - *Generally speaking, a polynomial of degree d (highest exponent) is* $O(n^d)$
  - 2^{n+2} *is* $\mathbf{O(2^n)}$
    - *Justification:*
      - $2^{n+2} = 2^n \times 2^2 = 4 \times 2^n = c2^n$, *for* $c = 4$ *and* $n_0 = 1$

Asymptotic Analysis (cont.)

- In general, ***we should use the big-Oh notation to characterize a function as closely as possible***.
  - Strictly speaking a function *f(n)* that is *O(n)* is, for example, also $O(n^2)$, $O(n^3)$, or even $O(2^n)$.

  - **We follow the** (textbook‘s and others) **practice** to say *f(n)* is *O(n)* because that is the „tightest“ upper-bound on its growth rate.
    - In the next course in the *Programming - Data Structures* – ***Algorithms*** course triology you will be introduced to a more precise notation:

      O(…) vs **Θ**(…) , Big-Oh vs Big-Theta , (as well as **Ω**(…))

      For courious minds, see also p. 127 in the textbook.

Asymptotic Analysis (cont.)

n	$\log n$	n	$n \log n$	n^2	n^3	2^n
8	3	8	24	64	512	256
16	4	16	64	256	4,096	65,536
32	5	32	160	1,024	32,768	4,294,967,296
64	6	64	384	4,096	262,144	1.84×10^{19}
128	7	128	896	16,384	2,097,152	3.40×10^{38}
256	8	256	2,048	65,536	16,777,216	1.15×10^{77}
512	9	512	4,608	262,144	134,217,728	1.34×10^{154}

Table 3.2: Selected values of fundamental functions in algorithm analysis.

Table from the textbook: *Data Structures & Algorithms*, Goodrich et al.

Asymptotic Analysis (cont.)

Running Time (μs)	Maximum Problem Size (n)		
	1 second	1 minute	1 hour
$400n$	2,500	150,000	9,000,000
$2n^2$	707	5,477	42,426
2^n	19	25	31

Table 3.3: Maximum size of a problem that can be solved in 1 second, 1 minute, and 1 hour, for various running times measured in microseconds.

Table from the textbook: *Data Structures & Algorithms*, Goodrich et al.

Asymptotic Analysis (cont.)

Running Time	New Maximum Problem Size
$400n$	$256m$
$2n^2$	$16m$
2^n	$m+8$

Table 3.4: Increase in the maximum size of a problem that can be solved in a fixed amount of time, by using a computer that is 256 times faster than the previous one. Each entry is a function of m, the previous maximum problem size.

Table from the textbook: *Data Structures & Algorithms*, Goodrich et al.

Asymptotic Analysis (cont.)

- Some words of caution:
  - The big-Oh notation can be somewhat misleading should the constant factor (which we ignore) be **very** large.
    - For example, running time $n \times log(n)$ is generally preferable to a running time $10^{100} \times n$, even though the latter is asymptotically faster, i.e. $O(n)$.
  - In most practical settings, algorithms with asymptotic time complexity:
    - constant / logarithmic: have great efficiency
    - linear / log-linear: have good efficiency
      - for example, most algorithms that process input sequences are at least O(n) Q: Why?
    - quadratic / cubic: have not-too-good efficiency
      - but often the best we can do
    - exponential: have really bad efficiency
      - but, unfortunately, sometimes the best we can do

Example (cont.)

The ***PrefixAvg*** function takes as an input an array *A* of integers:

10	20	30	40	50

and returns an array *PA* of *floats*:

10.0	15.0	20.0	25.0	30.0

where *PA[i]* is the average of *A[0]…A[i]*.

Q: Can you improve upon this naive Python implementation? Explain in your own words how you would go about that?

```python
#
# A naive implementation of PrefixAvg.
# What is its time-complexity? O(?)
# Can you write an improved version that is O(n)?
# Is it possible to do better time-complexity-wise?
#
def avg(l: list[int]) -> float:
    assert l, "List must be non-empty!"
    return sum(l) / len(l)

def prefix_avg(a: list[int]) -> list[float]:
    pa: list[float] = [0.0] * len(a)
    for i in range(len(a)):
        pa[i] = round(avg(a[:i+1]), 1)
    return pa

n = 10
random_arr = [random.randint( a: 0,  b: 10) for _ in range(n)]
print(random_arr)
print(prefix_avg(random_arr))
```

Summary

- The main takeaways are:
  - Be aware of the ***time-complexity*** of the algorithms you use/write.
  - What ***data structures*** we use when implementing different algorithms typically affects the time-complexity
  - Be familiar with the ***Big-Oh*** notation for describing time complexity: **O(…)**

Image created using *artlist.io*